



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student Name	Tyrael Lance Peralta
Section:	AN42
Professor:	Crisola Tan
Date:	10/4/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF file**.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?

Rule-Based Reasoning (RBR) was easier to implement because it only requires simple IF–THEN conditions. Each rule directly checks a fact and produces an action. CBR requires creating past cases, comparing similarities, retrieving the closest case, and revising the solution.

2. Which is more flexible in solving unstructured problems?

Case-Based Reasoning (CBR) is more flexible because it can use previous experiences to solve new or unusual problems, even when there is no exact rule for them. RBR works best when the problem conditions and solutions are clearly defined.

3. Which is better for situations with continuously changing data?

CBR is generally better because new cases can be added to the case base as situations change. This allows the system to learn from recent experiences. However, RBR can also handle changing data if its rules are regularly updated.

4. Examples of real-world applications for both.

Rule-Based Reasoning:

- Medical expert systems that identify diseases using symptoms
- Air-conditioning and smart-home control systems
- Bank fraud detection using fixed warning rules
- Troubleshooting systems for computers or machines

Case-Based Reasoning:

- Help-desk systems that use previous technical problems
- Medical diagnosis based on similar patient cases
- Legal systems that compare current cases with past court cases
- Car repair and mechanical troubleshooting

Your Code - Laboratory Tasks

Source Code:

Dictionary

```
facts = {
```

```
    # Category 1: Environment
```

```
    "temperature": 32,
```

```
    "is_raining": False,
```

```
    "humidity": 85,
```

```
    # Category 2: Appliance status
```

```
    "aircon_on": False,
```

```
    "fan_on": False,
```

```
    # Category 3: Presence and actions
```

```
    "person_present": True,
```

```
    "window_open": True
```

```
}
```

Rule Based Reasoning

Rule 1

```
def rule_hot(facts):
```

```
    if facts["temperature"] > 30:
```

```
        return "Turn on the aircon."
```

```
    return None
```

Rule 2

```
def rule_humid(facts):
```

```
    if facts["humidity"] > 80:
```

```
        return "Turn on the fan."
```

```
    return None
```

Rule 3

```

def rule_raining(facts):
    if facts["is_raining"]:
        return "Close the window."
    return None

# Rule 4
def rule_person_present(facts):
    if facts["person_present"] and not facts["aircon_on"]:
        return "Check the aircon."
    return None

# Inference engine
def inference_engine(facts, rules):
    actions = []

    for rule in rules:
        result = rule(facts)

        if result is not None:
            actions.append(result)

    return actions

rules = [
    rule_hot,
    rule_humid,
    rule_raining,
    rule_person_present
]

rbr_actions = inference_engine(facts, rules)

print("RULE-BASED REASONING ACTIONS:")
for action in rbr_actions:
    print("-", action)

# Case base containing previous problems and solutions
case_base = [
    {
        "problem": {
            "temperature_high": True,
            "humidity_high": True,
            "aircon_not_working": False
        },
    },

```

```

        "solution": "Turn on the aircon and fan."
    },
    {
        "problem": {
            "temperature_high": False,
            "humidity_high": True,
            "aircon_not_working": False
        },
        "solution": "Turn on the fan."
    },
    {
        "problem": {
            "temperature_high": True,
            "humidity_high": False,
            "aircon_not_working": True
        },
        "solution": "Check the aircon power cable."
    }
]

```

Current problem

```

new_problem = {
    "temperature_high": facts["temperature"] > 30,
    "humidity_high": facts["humidity"] > 80,
    "aircon_not_working": True
}

```

Retrieve Step

```

def calculate_similarity(problem1, problem2):
    matching_features = 0

    for feature in problem1:
        if feature in problem2 and problem1[feature] == problem2[feature]:
            matching_features += 1

    return matching_features

def retrieve_most_similar_case(problem, case_base):
    most_similar_case = None
    highest_similarity = -1

    for case in case_base:
        similarity = calculate_similarity(problem, case["problem"])

```

```

        if similarity > highest_similarity:
            highest_similarity = similarity
            most_similar_case = case

    return most_similar_case

# Reuse Step

similar_case = retrieve_most_similar_case(new_problem, case_base)

reused_solution = similar_case["solution"]

print("\nCASE-BASED REASONING:")
print("New problem:", new_problem)
print("Retrieved solution:", reused_solution)

# Revise Step

print("\nREVISE STEP")
print("Suggested solution:", reused_solution)

revision = input(
    "Enter a revised solution, or press Enter to keep the original: "
).strip()

if revision != "":
    final_solution = revision
else:
    final_solution = reused_solution

print("Final solution:", final_solution)

# Retain Step

new_case = {
    "problem": new_problem,
    "solution": final_solution
}

case_base.append(new_case)

print("\nRETAIN STEP")
print("New case has been added to the case base.")

```

```
print("Total number of cases:", len(case_base))
```

```
RULE-BASED REASONING ACTIONS:
```

- Turn on the aircon.
- Turn on the fan.
- Check the aircon.

```
CASE-BASED REASONING:
```

```
New problem: {'temperature_high': True, 'humidity_high': True, 'aircon_not_working': True}
```

```
Retrieved solution: Turn on the aircon and fan.
```

```
REVISE STEP
```

```
Suggested solution: Turn on the aircon and fan.
```

```
Enter a revised solution, or press Enter to keep the original: Turn on the fan only
```

```
Final solution: Turn on the fan only
```

```
RETAIN STEP
```

```
New case has been added to the case base.
```

```
Total number of cases: 4
```

```
(.venv) PS C:\Files\School\Fourth\IntelSystems\TechAssesments\TA1> █
```